



1. Creation of five child processes

Write a C program to create five child processes using system call `fork()` and display their process IDs.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int pid,i;
    printf("The parent process id=%d\n",getpid());
    for(i=1; i<=5; i++)
    {
        int pid=fork();
        if(pid<0)
        {
            printf("Fork failed!");
            exit(0);
        }
        else if(pid==0)
        {
            printf("Child process id=%d\n",getpid());
            exit(0);
        }
        wait(NULL);
    }
    return 0;
}
```

Note:

The first program can be executed only in linux environment. Because `fork()` is a linux system call. Windows can't able to recognize it. Other program can be executed in any environments as well as in codeblocks IDE

Output:

```
shawnlintonmiranda@HP:~/OS$ vi 1.c
shawnlintonmiranda@HP:~/OS$ cc 1.c
shawnlintonmiranda@HP:~/OS$ ./a.out
The parent process id=8669
Child process id=8670
Child process id=8671
Child process id=8672
Child process id=8673
Child process id=8674
```

1. FCFS Scheduling algorithm

Write a C program to implement First Come First Serve Scheduling algorithm to determine total waiting time, average waiting time, total turn around time and average turn around time.

Program:

```
#include <stdio.h>
struct process
{
    int pid,bt,wt,tt;
};
```

```

int main()
{
    int i,n,totwt=0,tottt=0;
    float avgwt,avgtt;
    printf("Enter the number of processes : ");
    scanf("%d",&n);
    struct process p[n]; //Allocate space
    for(i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("Enter burst time for process-%d : ", (i+1));
        scanf("%d",&p[i].bt);
    }
    p[0].wt=0;      //WT for 1st process is 0
    p[0].tt=p[0].bt+p[0].wt;      //BT for 1st process is wt+its BT
    for(i=1; i<n; i++)      //Remaining processes 2 to n
    {
        p[i].wt=p[i-1].bt+p[i-1].wt;      //OR p[i].wt=p[i-1].tt;
        p[i].tt=p[i].wt+p[i].bt;
    }
    printf("\nFCFS Scheduling\nProcess ID\tBT\tWT\tTAT\n");
    for(i=0; i<n; i++)
    {
        printf("\n\t%d\t%d\t%d\t%d",p[i].pid,p[i].bt,p[i].wt,p[i].tt);
        totwt=totwt+p[i].wt;
        tottt=tottt+p[i].tt;
    }
    avgwt=(float)totwt/n;
    avgtt=(float)tottt/n;
    printf("\n\nTotal Waiting Time : %dms\nAverage Waiting Time : %.2fms\n",totwt,avgwt);
    printf("\nTotal turn-around time : %dms\nAverage turn-around time : %.2fms\n",tottt,avgtt);
    return 0;
}

```

Output:

```

shawnlintonmiranda@HP:~/OS$ vi 2.c
shawnlintonmiranda@HP:~/OS$ cc 2.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter the number of processes : 3
Enter burst time for process-1 : 24
Enter burst time for process-2 : 3
Enter burst time for process-3 : 3

FCFS Scheduling
Process ID      BT      WT      TAT

      1      24      0      24
      2       3     24     27
      3       3     27     30

Total Waiting Time : 51ms
Average Waiting Time : 17.00ms

Total turn-around time : 81ms
Average turn-around time : 27.00ms

```

2. SJF Scheduling algorithm

Write a C program to implement Shortest Job First Scheduling algorithm to determine total waiting time, average waiting time, total turn around time and average turn around time.

Program:

```
#include <stdio.h>
struct process
{
    int pid,bt,wt,tt;
};
int main()
{
    int i,j,n,totwt=0,totlt=0;
    float avgwt,avgtt;
    printf("Enter the number of processes : ");
    scanf("%d",&n);
    struct process p[n],temp;
    for(i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("Enter burst time for process-%d : ",i+1);
        scanf("%d",&p[i].bt);
    }
    int min;
    for(i=0; i<n-1; i++) //Selection sort
    {
        for(j=i+1; j<n; j++)
        {
            min=i;
            if(p[j].bt<p[min].bt)
                min=j;
        }
        temp.pid=p[i].pid;
        temp.bt=p[i].bt;
        p[i].pid=p[min].pid;
        p[i].bt=p[min].bt;
        p[min].pid=temp.pid;
        p[min].bt=temp.bt;
    }
    p[0].wt=0; //Proceed same as FCFS after sorting
    p[0].tt=p[0].bt+p[0].wt;
    for(i=1; i<n; i++)
    {
        p[i].wt=p[i-1].bt+p[i-1].wt;      //OR p[i].wt=p[i-1].tt;
        p[i].tt=p[i].wt+p[i].bt;
    }
    printf("\nSJF Scheduling\nProcess ID\tBT\tWT\tTAT\n");
    for(i=0; i<n; i++)
    {
        printf("\n\t%d\t%d\t%d\t%d",p[i].pid,p[i].bt,p[i].wt,p[i].tt);
        totwt=totwt+p[i].wt;
        totlt=totlt+p[i].tt;
    }
}
```

```

avgwt=(float)totwt/n;
avgtt=(float)tottt/n;
printf("\n\nTotal Waiting Time : %dms\nAverage Waiting Time : %.2fms\n",totwt,avgwt);
printf("\nTotal turn-around time : %dms\nAverage turn-around time : %.2fms\n",tottt,avgtt);
return 0;
}

```

Output:

```

shawnlintonmiranda@HP:~/0S$ vi 3.c
shawnlintonmiranda@HP:~/0S$ cc 3.c
shawnlintonmiranda@HP:~/0S$ ./a.out
Enter the number of processes : 4
Enter burst time for process-1 : 6
Enter burst time for process-2 : 8
Enter burst time for process-3 : 7
Enter burst time for process-4 : 3

SJF Scheduling
Process ID      BT      WT      TAT
      4         3       0       3
      1         6       3       9
      3         7       9      16
      2         8      16      24

Total Waiting Time : 28ms
Average Waiting Time : 7.00ms

Total turn-around time : 52ms
Average turn-around time : 13.00ms

```

3.Round Robin Scheduling algorithm

Write a C program to implement Round Robin Scheduling algorithm to determine total waiting time, average waiting time, total turn around time and average turn around time.

Program:

```

#include <stdio.h>
struct process {
    int pid,bt,wt,tt;
};
int main()
{
    int i,n,tq,time=0,totwt=0,tottt=0,flag=0;
    float avgwt,avgtt;
    printf("Enter the number of processes : ");
    scanf("%d",&n);
    struct process p[n];
    int remBT[n];
    for(i=0; i<n; i++)
    {
        p[i].pid=i+1;
        printf("Enter burst time for process-%d : ",i+1);
        scanf("%d",&p[i].bt);
        remBT[i]=p[i].bt;
    }
    printf("\nEnter the time quantum : ");

```

```

scanf("%d",&tq);
while(flag==0)
{
    flag=1; //Assume all completed execution
    for(i=0; i<n; i++){
        if(remBT[i]>0)
        {
            flag=0; // Some processes are remaining
            if(remBT[i]>tq) {
                time=time+tq;
                remBT[i]=remBT[i]-tq;
            }
            else {
                time=time+remBT[i];
                remBT[i]=0;
                p[i].tt=time;
                p[i].wt=p[i].tt-p[i].bt;
            }
        }
    }
}

printf("\nRound Robin Scheduling\nProcess ID\tBT\tWT\tTAT\n");
for(i=0; i<n; i++) {
    printf("\n\t%d\t%d\t%d\t%d",p[i].pid,p[i].bt,p[i].wt,p[i].tt);
    totwt=totwt+p[i].wt;
    tottt=tottt+p[i].tt;
}
avgwt=(float)totwt/n;
avgtt=(float)tottt/n;
printf("\n\nTotal Waiting Time : %dms\nAverage Waiting Time : %.2fms\n",totwt,avgwt);
printf("\nTotal turn-around time : %dms\nAverage turn-around time : %.2fms\n",tottt,avgtt);
return 0;
}

```

Output:

```

shawnlintonmiranda@HP:~/OS$ vi 4.c
shawnlintonmiranda@HP:~/OS$ cc 4.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter the number of processes : 4
Enter burst time for process-1 : 3
Enter burst time for process-2 : 6
Enter burst time for process-3 : 4
Enter burst time for process-4 : 2

Enter the time quantum : 2

Round Robin Scheduling
Process ID      BT      WT      TAT
      1         3       6       9
      2         6       9      15
      3         4       9      13
      4         2       6       8

Total Waiting Time : 30ms
Average Waiting Time : 7.50ms

Total turn-around time : 45ms
Average turn-around time : 11.25ms

```

4. Producer Consumer problem

Write a C program to simulate producer – consumer problem using semaphores.

Program:

```
#include<stdio.h>
int main()
{
    int bufferSize,in=0,out=0,items=0,produce,consume,choice=0;
    printf("Enter the buffer size : ");
    scanf("%d",&bufferSize);
    int buffer[bufferSize];
    do {
        printf("\n1.Procedure\t2.Consume\t3.Exit\n");
        printf("Enter your choice : ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:if(items==bufferSize)
                    printf("\nBuffer is full!");
                else {
                    printf("Enter the item value : ");
                    scanf("%d",&produce);
                    buffer[in]=produce;
                    in=(in+1)%bufferSize;
                    items++;
                }
                break;
            case 2:if(items==0)
                    printf("\nBuffer is empty!");
                else {
                    consume=buffer[out];
                    printf("\nThe consumed item value is %d.",consume);
                    out=(out+1)%bufferSize;
                    items--;
                }
                break;
        }
    } while(choice!=3);
    return 0;
}
```

Output:

```
shawnlintonmiranda@HP:~/OS$ vi 5.c
shawnlintonmiranda@HP:~/OS$ cc 5.c
shawnlintonmiranda@HP:~/OS$ ./a.out
```

Enter the buffer size : 2

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 1
Enter the item value : 10
```

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 1
Enter the item value : 20
```

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 1
```

Buffer is full!

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 2
```

The consumed item value is 10.

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 2
```

The consumed item value is 20.

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 2
```

Buffer is empty!

```
1.Procedure      2.Consume      3.Exit
Enter your choice : 3
```

5. FIFO Page Replacement algorithm

Write a C program to demonstrate First In First Out page replacement algorithm to determine number of page faults.

Program:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n,f,i,j,found,victim=-1,pf=0;
    printf("Enter the number of pages : ");
    scanf("%d",&n);
    int ref[n];
    printf("Enter number of frames : ");
    scanf("%d",&f);
    int frame[f];
    for(j=0; j<f; j++)
        frame[j]=-1;
    printf("Enter the reference string : ");
    for(i=0; i<n; i++)
        scanf("%d",&ref[i]);

    printf("\nFIFO Page Replacement\nReference String is \n");
    for(i=0; i<n; i++)
        printf("%d ",ref[i]);
    printf("\n\n");
    for(i=0; i<n; i++)
    {
        found=0; //Assume page is not found
        for(j=0; j<f; j++) //Search for page
            if(frame[j]==ref[i])
            {
                found=1; //Page found
                break;
            }
        if(!found) //Or use found==0
        {
            pf++;
            victim=(victim+1)%f;
            frame[victim]=ref[i];
        }
        printf("%d -> ",ref[i]);
        for(j=0; j<f; j++)
            printf(" %d ",frame[j]);
        if(found==0) printf("\tPage miss\n");
        else printf("\tPage hit\n");
    }
    printf("\nTotal number of page faults : %d\n",pf);
    return 0;
}
```

Output:

```
shawnlintonmiranda@HP:~/OS$ vi 6.c
shawnlintonmiranda@HP:~/OS$ cc 6.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter the number of pages : 12
Enter number of frames : 4
Enter the reference string : 1 2 3 4 1 2 5 1 2 3 4 5

FIFO Page Replacement
Reference String is
1  2  3  4  1  2  5  1  2  3  4  5

1 -> 1  -1  -1  -1      Page miss
2 -> 1  2  -1  -1      Page miss
3 -> 1  2  3  -1      Page miss
4 -> 1  2  3  4      Page miss
1 -> 1  2  3  4      Page hit
2 -> 1  2  3  4      Page hit
5 -> 5  2  3  4      Page miss
1 -> 5  1  3  4      Page miss
2 -> 5  1  2  4      Page miss
3 -> 5  1  2  3      Page miss
4 -> 4  1  2  3      Page miss
5 -> 4  5  2  3      Page miss

Total number of page faults : 10
```

6. LRU Page Replacement algorithm

Write a C program to demonstrate Least Recently Used page replacement algorithm to determine number of page faults.

Program:

```
#include<stdio.h>

void main() {
    int n,f,i,j,free,min,index,found,pos=0,time=0,pf=0;
    printf("Enter the number of pages : ");
    scanf("%d",&n);
    int ref[n];
    printf("Enter number of frames : ");
    scanf("%d",&f);
    free=f;
    int frame[f],count[f];
    for(j=0; j<f; j++) {
        frame[j]=-1;
        count[j]=0;
    }
    printf("Enter the reference string : ");
    for(i=0; i<n; i++)
        scanf("%d",&ref[i]);
    for(i=0; i<n; i++) {
        found=0;    //Assume page not found
        for(j=0; j<f; j++) {
            if(frame[j]==ref[i])
            {
                count[j]=time;
                found=1; //page found
                break;
            }
        }
    }
}
```



```

        if(!found) {
            if(free) {
                frame[pos]=ref[i];
                pf++;
                free--;
                count[pos]=time;
                pos++;
            }
            else {
                min=count[0];
                index=0; //Assume 1st frame is LRU
                for(j=1;j<f;j++) //Find other min
                    if(count[j]<min) {
                        min=count[j];
                        index=j;
                    }
                frame[index]=ref[i];
                count[index]=time;
                pf++;
            }
        }
        time++;
        printf("%d -> ",ref[i]);
        for(j=0; j<f; j++)
            printf(" %d ",frame[j]);
        if(found==0) printf("\tPage miss\n");
        else printf("\tPage hit\n");
    }
    printf("\nNo of page faults:%d\n",pf);
}

```

Output:

```

shawnlintonmiranda@HP:~/OS$ vi 7.c
shawnlintonmiranda@HP:~/OS$ cc 7.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter the number of pages : 20
Enter number of frames : 3
Enter the reference string : 1 2 3 5 2 3 5 7 2 1 2 3 8 6 4 3 2 2 3 6
1 -> 1  -1  -1      Page miss
2 -> 1  2   -1      Page miss
3 -> 1  2   3       Page miss
5 -> 5  2   3       Page miss
2 -> 5  2   3       Page hit
3 -> 5  2   3       Page hit
5 -> 5  2   3       Page hit
7 -> 5  7   3       Page miss
2 -> 5  7   2       Page miss
1 -> 1  7   2       Page miss
2 -> 1  7   2       Page hit
3 -> 1  3   2       Page miss
8 -> 8  3   2       Page miss
6 -> 8  3   6       Page miss
4 -> 8  4   6       Page miss
3 -> 3  4   6       Page miss
2 -> 3  4   2       Page miss
2 -> 3  4   2       Page hit
3 -> 3  4   2       Page hit
6 -> 3  6   2       Page miss

No of page faults:14

```

X 8. Optimal Page Replacement algorithm

Write a C program to demonstrate Optimal page replacement algorithm to determine number of page faults.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int n,f;
int optimal(int ref[],int frame[],int optindex[],int index)
{
    int pos,found,max,i,j;
    for(i=0;i<f;i++)
    {
        found=0;    //Assume page not found
        for(j=index+1;j<n;j++)
        {
            if(frame[i]==ref[j])
            {
                found=1; //Page found
                optindex[i]=j;
                break;
            }
        }
        if(!found)
            return i;
    }
    max=optindex[0];
    pos=0;
    for(i=1;i<f;i++)
    {
        if(max<optindex[i])
        {
            max=optindex[i];
            pos=i;
        }
    }
    return pos;
}
int main()
{
    int i,j,found,count=0,pf=0,victim=-1;
    printf("Enter the number of pages : ");
    scanf("%d",&n);
    int ref[n];
    printf("Enter number of frames : ");
    scanf("%d",&f);
    int frame[f],optindex[f];
    for(j=0; j<f; j++) {
        frame[j]=-1;
        optindex[j]=-1;
    }
    printf("Enter the reference string : ");
    for(i=0; i<n; i++)
        scanf("%d",&ref[i]);
```

```

for(i=0;i<n;i++)
{
    found=0; //Assume page not found.
    for(j=0;j<f;j++)
    {
        if(ref[i]==frame[j]) {
            found=1; //Page found
            break;
        }
    }
    if(!found)
    {
        count++;
        if(count<=f)
            victim++;
        else
            victim=optimal(ref,frame,optindex,i);
        pf++;
        frame[victim]=ref[i];
    }
    printf("%d -> ",ref[i]);
    for(j=0; j<f; j++)
        printf(" %d ",frame[j]);
    if(found==0) printf("\tPage miss\n");
    else printf("\tPage hit\n");
}
printf("\nNumber of page fault:%d\n",pf);
}

```

Output:

```

Number of page fault:10shawnlintonmiranda@HP:~/OS$ cc 8.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter the number of pages : 20
Enter number of frames : 3
Enter the reference string : 1 2 3 5 2 3 5 7 2 1 2 3 8 6 4 3 2 2 3 6
1 -> 1 -1 -1 Page miss
2 -> 1 2 -1 Page miss
3 -> 1 2 3 Page miss
5 -> 5 2 3 Page miss
2 -> 5 2 3 Page hit
3 -> 5 2 3 Page hit
5 -> 5 2 3 Page hit
7 -> 7 2 3 Page miss
2 -> 7 2 3 Page hit
1 -> 1 2 3 Page miss
2 -> 1 2 3 Page hit
3 -> 1 2 3 Page hit
8 -> 8 2 3 Page miss
6 -> 6 2 3 Page miss
4 -> 4 2 3 Page miss
3 -> 4 2 3 Page hit
2 -> 4 2 3 Page hit
2 -> 4 2 3 Page hit
3 -> 4 2 3 Page hit
6 -> 6 2 3 Page miss
Number of page fault:10

```

9. Bankers Algorithm for deadlock avoidance

Write a C program to demonstrate Bankers Deadlock avoidance algorithm.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int p,r,i,j,k,found=0,count=0;
    printf("Enter number of processes:\n");
    scanf("%d",&p);
    printf("Enter number of resources:\n");
    scanf("%d",&r);
    int alloc[p][r],max[p][r],avail[r],need[p][r],safeseq[p],finish[p];
    printf("\nEnter the allocation matrix:\n");
    for(i=0;i<p;i++)
        for(j=0;j<r;j++)
            scanf("%d",&alloc[i][j]);
    printf("\nEnter the max matrix:\n");
    for(i=0;i<p;i++)
        for(j=0;j<r;j++)
            scanf("%d",&max[i][j]);
    printf("\nEnter the available matrix:\n");
    for(i=0;i<r;i++)
        scanf("%d",&avail[i]);
    for(i=0;i<p;i++)
        for(j=0;j<r;j++)
            need[i][j]=max[i][j]-alloc[i][j];
    printf("\nNeed Matrix :\n");
    for(i=0;i<p;i++)
    {
        for(j=0;j<r;j++)
            printf("%d\t",need[i][j]);
        printf("\n");
    }
    for(i=0;i<p;i++)
        finish[i]=0;
    while(count<p)
    {
        found=0; //Assume !(need<=available) for all processes
        for(i=0;i<p;i++)
        {
            if(finish[i]==0) //Didn't completed
            {
                for(j=0;j<r;j++)
                {
                    if(need[i][j]>avail[j])
                        break;
                }
                if(j==r) //need<=available
                {
                    for(k=0;k<r;k++)
                        avail[k]=avail[k]+alloc[i][k];
                    safeseq[count++]=i;
                    found=1; //need<=available found
                    finish[i]=1; //Mark process as finished
                }
            }
        }
    }
}
```

```

    }
    if(found==0) //No process can be allocated.
    {
        printf("System is in unsafe state.\n");
        return 0;
    }
}
printf("System is in safe state.\n");
printf("Safe Sequence is : ");
for(k=0;k<p;k++)
    printf("P%d\t",safeseq[k]+1);
printf("\n");
return 0;
}

```

Output:

```

shawnlintonmiranda@HP:~/OS$ vi 9.c
shawnlintonmiranda@HP:~/OS$ cc 9.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter number of processes:
5
Enter number of resources:
4

Enter the allocation matrix:
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4

Enter the max matrix:
0 0 1 2
1 7 5 0
2 3 5 6
0 6 5 2
0 6 5 6

Enter the availble matrix:
1 5 2 0

Need Matrix :
0      0      0      0
0      7      5      0
1      0      0      2
0      0      2      0
0      6      4      2
System is in safe state.
Safe Sequence is : P1      P3      P4      P5      P2

```

```

shawnlintonmiranda@HP:~/OS$ cc 9.c
shawnlintonmiranda@HP:~/OS$ ./a.out
Enter number of processes:
2
Enter number of resources:
2

Enter the allocation matrix:
0 1
1 0

Enter the max matrix:
3 4
4 5

Enter the availble matrix:
1 1

Need Matrix :
3      3
3      5
System is in unsafe state.

```

